

Finly Engineering Documentation

Document Control

Field	Value
Document type	System/Engineering documentation
Status	Active
Version	1.1
Last updated	2026-04-16
Primary audience	Developers, reviewers, DevOps, maintainers
Repository	FinanceTracker

Table of Contents

1. Purpose and Scope
2. Product Summary
3. System Context
4. Architecture
5. Data Model (High-Level)
6. API Contract Standards
7. Security Controls
8. Configuration Management
9. Build, Run, and Deployment
10. Observability and Operations
11. Recovery and Continuity Notes
12. Testing and Quality Expectations
13. Known Constraints
14. Maintainer Runbooks
15. References
16. Non-Functional Requirements (NFRs)
17. Release and Change Management
18. Incident Management Model

1) Purpose and Scope

This document defines the technical architecture, runtime behavior, operations model, and maintenance standards for Finly.

In scope:

- backend API architecture and contracts
- frontend application structure
- security and access controls
- configuration and deployment model
- operational runbooks and troubleshooting

Out of scope:

- business policy decisions
- financial/accounting compliance interpretation

2) Product Summary

Finly is a role-based finance tracking platform with user and admin workspaces.

- Users manage transactions, budgets, reports, and profile settings.
- Admins monitor platform usage, users, categories, analytics, and logs.
- API is versioned under `/api/v1` and serves a static frontend from FastAPI.

3) System Context

Core runtime components

- Backend: FastAPI app (`app/main.py`)
- Data layer: SQL Server via SQLAlchemy + ODBC driver 18
- Frontend: static HTML/CSS/JS in `Frontend/`
- Auth: JWT bearer token and role checks
- Email/reset: SMTP (Gmail-compatible) with persistent reset code store

Deployment topology (current production)

- Hosting: Azure Container Apps (Consumption)
- Database: Azure SQL Database
- Image registry: Docker Hub
- Reference deployment playbook: `docs/azure-deploy.md`

4) Architecture

4.1 Backend architecture

- Entry point: `app/main.py`
- Router modules:
 - `auth_router (/auth)`
 - `user_router (/users)`
 - `category_router (/categories)`
 - `transaction_router (/transactions)`
 - `budget_router (/budgets)`
 - `analytics_router (/summary)`
 - `admin_router (/admin)`
 - `log_router (/logs)`
- API mounted under `/api/v1`.
- Legacy route compatibility middleware rewrites legacy endpoints to `/api/v1`.

4.2 Middleware and cross-cutting behavior

- RequestIDMiddleware: injects `X-Request-ID`, `X-Process-Time`
- API cache policy middleware: no-store/no-cache headers for API routes
- CORS middleware: uses `CORS_ALLOWED_ORIGINS`
- Rate limiter middleware: request throttling baseline

4.3 Frontend architecture

- Public pages: `index.html`, `support.html`, auth pages
- Workspace shells: `app.html` (user), `admin.html` (admin)
- JS modules in `Frontend/scripts/`
- Shell controller: `Frontend/scripts/workspace.js`
- Page-specific logic: `Frontend/scripts/pages/user.js`, `Frontend/scripts/pages/admin.js`

5) Data Model (High-Level)

Main entities:

- **users**: identity, role, status, password hash
- **categories**: global category master (admin-managed)
- **transactions**: user-linked financial entries (soft-delete aware)
- **budgets**: per-user budget plans by period
- **audit_log**: operational history/events
- **password_reset_codes**: persistent reset-code store with expiry/attempt controls

Data ownership/RBAC:

- Non-admin users access only their own transaction/budget domain.
- Admin users can access privileged admin endpoints and global operational views.

6) API Contract Standards

6.1 Versioning

- Canonical base path: `/api/v1`
- `/api` and legacy prefixes are redirected/re-written to preserve compatibility.

6.2 Authentication and authorization

- Authentication: JWT bearer tokens (`Authorization: Bearer <token>`)
- Authorization: role check via `require_admin` for admin routes

6.3 Response model

- Success responses follow a structured envelope (e.g., `success`, `data`, `meta`).

- Error responses are normalized by centralized exception handlers.

6.4 Core endpoints (representative)

- Health: GET /health
- Auth: /api/v1/auth/*
- User/Profile: /api/v1/users/*
- Transactions: /api/v1/transactions/*
- Budgets: /api/v1/budgets/*
- Categories: /api/v1/categories/*
- Analytics: /api/v1/summary/*
- Admin: /api/v1/admin/*
- Logs: /api/v1/logs/*

7) Security Controls

- Password hashing: bcrypt
- JWT signed with JWT_SECRET (HS256)
- Admin endpoint protection: explicit dependency guard
- Rate limiting:
 - login/signup throttling
 - password reset request/attempt throttling
- CORS policy enforced via explicit origin configuration
- Secrets managed as environment variables / cloud secrets

Operational security guidance:

- Never commit secrets to source control.
- Prefer managed secret stores for production (`az containerapp secret set`).
- Keep `DEBUG_RESET_CODES=0` in production unless temporary diagnostics are required.

8) Configuration Management

Baseline config file: `.env.example`

Required production keys:

- `APP_ENV=production`
- `DB_SERVER`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`
- `DB_ENCRYPT=yes`, `DB_TRUST_SERVER_CERTIFICATE=no` (recommended for Azure SQL)
- `JWT_SECRET`
- `CORS_ALLOWED_ORIGINS`

Email/recovery keys:

- `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASSWORD`, `FROM_EMAIL`

- optional: BREVO_API_KEY flow (fallback/alternative)

9) Build, Run, and Deployment

9.1 Local development

- Start stack: `make dev`
- Initialize schema: `make init-db`
- Share local app: `make share`

9.2 Production deployment

- Current production pattern: Docker image -> Azure Container Apps + Azure SQL
- Detailed command history and exact resource names: `docs/azure-deploy.md`

10) Observability and Operations

10.1 Health and diagnostics

- Liveness endpoint: `GET /health`
- Request tracing via `X-Request-ID`
- Application logs available through Container Apps log stream

10.2 Runtime troubleshooting checklist

1. Confirm container revision health (`az containerapp revision list`).
2. Verify DB connectivity and credentials (`db-password secret sync`).
3. Check API health and static route responses.
4. Validate auth flow (`/api/v1/auth/login`).
5. Inspect logs for SMTP/reset and SQL errors.

11) Recovery and Continuity Notes

- Primary persistent store is SQL Server (transactions/budgets/users/logs).
- Password reset codes are persisted via local SQLite file path configured by `PASSWORD_RESET_STORE_PATH`; for horizontally scaled production, use a shared persistent approach if strict cross-replica consistency is required.

12) Testing and Quality Expectations

Minimum change validation before release:

- API smoke checks: health, auth login, key workspace APIs
- UI smoke checks: login, user workspace, admin workspace, exports
- DB checks: schema initialized, critical tables available
- Security checks: CORS origins, JWT secret present, no debug secrets exposed

Recommended CI gates:

- static linting/format checks
- test suite execution
- container build validation (`linux/amd64`)

13) Known Constraints

- Frontend is server-served static assets (no separate CDN pipeline yet).
- Azure Container Apps may briefly serve stale HTML after revision rollout until cache refresh.
- SMTP provider restrictions (app passwords, sender policy) can block delivery if misconfigured.

14) Maintainer Runbooks

14.1 Cannot login

- validate user exists/status in DB
- verify JWT secret consistency across revision
- test `/api/v1/auth/login` directly

14.2 Forgot-password not delivering email

- verify SMTP env and secret references
- inspect logs for SMTP authentication or provider errors
- temporarily enable `DEBUG_RESET_CODES=1` only for controlled diagnostics

14.3 Empty dashboard for new environment

- ensure categories exist (income + expense)
- seed demo records for validation users

15) References

- `README.md`
- `docs/PROJECT_DOCUMENTATION.md`
- `docs/API_CONTRACT.md`
- `docs/azure-deploy.md`
- `app/main.py`
- `app/core/auth.py`
- `app/core/config.py`

16) Non-Functional Requirements (NFRs)

16.1 Availability and reliability targets

- Service health endpoint (`/health`) must remain available for platform probes.
- Deployments should use zero-downtime revision switchovers where supported by platform.
- Production secrets rotation must trigger new revision rollout and post-rotation smoke checks.

16.2 Performance baselines

- API endpoints should return predictable response times under normal usage.
- Admin/log endpoints with elevated latency are explicitly tracked by middleware warnings.
- Query paths should maintain pagination support (`limit`, `offset`) to avoid unbounded payloads.

16.3 Maintainability

- API contracts must remain backward-compatible under `/api/v1` unless explicitly versioned.
- New endpoints must follow existing response envelope and centralized error model.
- Operational changes must be documented in `docs/azure-deploy.md` when deployment behavior changes.

16.4 Security and compliance posture

- Least-privilege access controls enforced by role checks.
- Passwords are never stored in plaintext; bcrypt hash only.
- Runtime secrets must remain externalized from source code.

17) Release and Change Management

17.1 Branch and release flow

- Feature work is developed in topic branches.
- Changes are validated via local smoke checks before merge/promotion.
- Production deploy uses immutable image digests for deterministic rollouts.

17.2 Pre-release checklist

1. Confirm app health (`/health`) and auth login path.
2. Validate user workspace and admin workspace rendering.
3. Verify DB schema compatibility and migration/init steps.

4. Verify SMTP/password reset behavior for current environment.
5. Confirm no debug-only security flags are left enabled.

17.3 Post-release checklist

1. Validate active revision health and traffic split.
2. Run endpoint smoke checks (`/`, `/login.html`, `/api/v1/auth/login`).
3. Review logs for startup/database/auth anomalies.
4. Capture deployment metadata in `docs/azure-deploy.md` if operationally significant.

18) Incident Management Model

18.1 Severity classification

Severity	Definition	Example
Sev-1	Complete outage or critical auth/data failure	API unavailable, login impossible for all users
Sev-2	Major feature degradation with workaround limits	admin analytics failing, reset flow broken
Sev-3	Partial/non-critical defects	UI rendering issue with working fallback
Sev-4	Cosmetic/documentation issues	copy or layout polish issue

18.2 Response expectations

- Sev-1: immediate mitigation, freeze non-essential deployments.
- Sev-2: prioritize same-day fix or safe rollback.
- Sev-3/4: schedule into normal backlog with documented workaround.

18.3 Standard incident workflow

1. Detect and classify severity.
2. Stabilize service (rollback or hotfix).
3. Validate core flows (health, auth, DB, key UI routes).
4. Communicate impact and status updates.
5. Record root cause and preventive actions in `docs/changelog`.